

CP468 Assignment 2: Connect Four AI Report

1. Introduction

This report details the implementation and evaluation of three distinct artificial intelligence agents designed to play the classic adversarial game, Connect Four. The objective of this project is to apply foundational AI techniques like heuristic evaluation and adversarial search to evaluate how increasing agent complexity impacts both game performance (win rates) and computational efficiency (decision time). The agents evaluated are a Random Agent (baseline), a manually-prioritized Rule-Based Agent, and a Minimax Agent utilizing Alpha-Beta pruning.

2. System Design

Game Engine Architecture

The underlying game engine represents the Connect Four board as a 6x7 two-dimensional list (array), where 0 represents an empty cell, 1 represents Player 1, and 2 represents Player 2. The top of the board is row 0, and the bottom is row 5.

Key engine mechanics include:

- Gravity Simulation: The `apply_move` function iterates from the bottom row (row 5) upwards to find the first available EMPTY cell in a selected column, accurately simulating gravity.
- Win Detection: The `is_winner` function scans the board upon every move, checking all 4 possible axes (horizontal, vertical, diagonal down-right, and diagonal down-left) for 4 consecutive identical tokens.
- Agent Integration: The engine exposes a clean interface (`legal_moves`, `apply_move`, `is_terminal`, `is_winner`) that all agents utilize. To prevent agents from accidentally mutating the actual game state during look-ahead searches, a `simulate_move` wrapper is utilized, which relies on `copy.deepcopy()` to safely test potential futures.

3. AI Agents

To ensure reliable and reproducible experimental data, a mandatory tie-breaking rule was implemented. Whenever an agent identifies multiple moves of equal value, it uses a pseudo-random number generator (seeded at 42) to choose uniformly among them.

Agent 1: Random Agent

This agent serves as the baseline for the experiment. It generates a list of all currently available non-full columns using `legal_moves()` and selects one uniformly at random using Python's `random.choice()`.

Agent 2: Rule-Based Agent

This agent relies on manually defined heuristics, executing the first rule that triggers in the following strict priority order:

1. Immediate Win: Uses `find_winning_moves` to check if a move will connect four discs for itself.
2. Block Opponent: Simulates the opponent's next turn to see if the opponent has an immediate winning move, and plays in that column to block it.
3. Center Preference: Calculates the distance of all valid moves from the center column (col 3) and isolates the moves closest to the center.
4. Extend Longest Line: Out of the central-most moves, it uses `_longest_line_through_cell` to simulate dropping a piece and counts which move creates the longest contiguous line of its own discs across any axis.
Tie-breaking: If multiple moves result in the same maximum line length under Rule 4, it picks uniformly at random between them.

Agent 3: Minimax Agent

This agent utilizes the Minimax algorithm with Alpha-Beta pruning to search the game tree.

- Search Depth: The agent is configured to a fixed look-ahead depth of 4.
- Terminal Evaluation: If a node is a terminal state, it assigns $10,000,000 + \text{depth}$ for a win, and $-10,000,000 - \text{depth}$ for a loss. Adding/subtracting the depth ensures the agent prefers faster wins and slower losses. Draws evaluate to 0.
- Heuristic Evaluation: At the depth limit (4), non-terminal states are scored using a "Windowed Scoring" technique. The algorithm slices the board into every possible length-4 window (horizontally, vertically, and diagonally). It evaluates each window based on the following weights:
 - 4 own discs: +100,000
 - 3 own discs (1 empty): +100
 - 2 own discs (2 empty): +10
 - 3 opponent discs (1 empty): -120 (Slightly higher penalty to prioritize defensive blocking)
 - 2 opponent discs (2 empty): -10
 - Center Control: An additional 6 points are awarded for every disc the agent owns in the absolute center column.

4. Experimental Results

Hardware & Environment Specifications:

- OS: Windows 11
- CPU: Intel Core i5-9500
- RAM: 16 GB @ 2400 MHz
- Language: Python 3.14.6
- Random Seed: 42

Protocol: 30 games were played per pairing, with the starting player alternating after 15 games.

Matchup	Wins	Draws	Avg. Decision Time (s)
Random vs. Rule-Based	Random: 0 (0.0%)	0 (0.0%)	Random: 0.0001s
	Rule-Based: 30 (100.0%)		Rule-Based: 0.0015s
Rule-Based vs. Minimax	Rule-Based: 8 (26.6%)	0 (0.0%)	Rule-Based: 0.0016s
	Minimax: 22 (73.3%)		Minimax: 0.085s
Minimax vs. Random	Minimax: 30 (100%)	0 (0.0%)	Minimax: 0.081s
	Random: 0 (0%)		Random: 0.0001s

The Minimax agent had the strongest performance with the highest win rate but the longest computation time, The Rule-Based Agent was strong against simple opponents while remaining computationally efficient, so overall a good all-arounder and The Random Agent was the fastest but weakest due to making uninformed moves. This means the more sophisticated search produces significantly better gameplay at the expense of longer decision times.

5. Discussion

The experimental evaluation reveals clear distinctions in how computational complexity translates to game performance.

As expected, the Minimax Agent exhibited the strongest overall performance, achieving a 100% win rate against the Random baseline and an 73.3% win rate against the Rule-Based agent. However, this dominance comes at a severe computational cost. Its average decision time (~0.08s) is exponentially higher than the other agents.

The Rule-Based Agent serves as an excellent middle-ground. It proved highly effective against simple opponents (100% win rate vs Random) while remaining computationally trivial (0.0015s per move). Interestingly, it managed to steal 8 wins against the Minimax agent. This likely occurs due to the "Horizon Effect" because Minimax is limited to a depth of 4, the Rule-Based agent occasionally creates traps that fall just outside Minimax's 4-turn predictive window.

The Random Agent was exceptionally fast but predictably weak, making entirely uninformed moves. The results clearly demonstrate that deeper, more sophisticated adversarial search produces significantly better gameplay, but scales poorly regarding time complexity.

Proposed AI Technique Improvements

To improve the agents further, the following techniques could be implemented:

1. **Deeper Search via Iterative Deepening:** Instead of a fixed depth of 4, the Minimax agent could use Iterative Deepening Search paired with a time limit (e.g., 2 seconds per move). This would allow it to search to depths of 6 or 8 in the early game when the branching factor is lower, eliminating the horizon effect that the Rule-based agent exploited.
2. **Transposition Tables (Memoization):** Many move sequences in Connect Four lead to the exact same board state. Implementing a transposition table (using Zobrist hashing) would allow the Minimax agent to cache previously evaluated board states, drastically reducing the number of nodes expanded and speeding up search times.
3. **Monte Carlo Tree Search (MCTS):** Replacing Minimax with MCTS would remove the need to manually tune heuristic weights entirely. MCTS would evaluate states by simulating hundreds of random playouts to the end of the game, which is often more robust in handling complex branching paths.
4. **Move Ordering:** The current Minimax alpha-beta implementation checks moves from left to right (column 0 to 6). By ordering the move generation to check the center columns first (3, then 2 and 4, etc.), alpha-beta pruning would find optimal cuts much earlier in the tree, improving efficiency.

6. Team Member Contributions

Team Member	Contribution
Jake Finkelstein	Game Engine
Ben Jones	AI Agents
Calista Lovelin Gino	Demonstration Video
Jonas Mogilnicki	Report
Jordan Strand	Experimental Evaluation